Artificial Intelligence and Robotics

A Complete Course in Plain English

ARTIFICIAL INTELLIGENCE AND ROBOTICS

Volume Three

Machine Learning

TBUGE MECHATRONIX

Artificial Intelligence and Robotics, Volume Three: Machine Learning

First edition. TBUGE Mechatronix.

This volume assumes the arithmetic of secondary school and the ideas of Volume One. It teaches the methods with a pencil first and a computer second, on the principle that a model you cannot compute by hand is a model you cannot debug.

The programme runs to forty-eight volumes. Volume One asked what intelligence is; Volume Two built the machine. This one gives that machine the ability to improve from examples.

Course codes have been left out on purpose. A subject should be named, not numbered.

Contents

Preface **6**

How This Volume Is Arranged **7**

Part One · The Shape of Learning **8**

The Shape of a Learning Problem 9

Data: Where It Comes From and What It Costs 15

Measuring Error 19

Part Two · Four Methods Worth Knowing **24**

A Straight Line, Fitted Properly 25

From a Number to a Probability 31

Neighbours, and the Trouble With Many Dimensions 36

Trees, Forests, and the Art of Asking Questions 40

Part Three · The Discipline of Not Fooling Yourself **44**

Overfitting, and the Two Kinds of Being Wrong 45

Validation, and the Many Ways to Cheat 50

Beyond Accuracy 55

Part Four · From Notebook to System **61**

Features and the Pipeline That Produces Them 62

Deployment, Drift and the Long Middle 66

When Machine Learning Is the Wrong Tool 72

Glossary **77**

Sources and Further Reading **80**

Preface

There is a moment in every machine learning project when the model reports a number and the team celebrates. The celebration is usually premature, and the reason is almost never the mathematics. It is that nobody asked what the number measured, on which data, chosen by whom, compared with what.

This volume is about that question. It teaches the standard methods of supervised and unsupervised learning — regression, classification, nearest neighbours, trees, ensembles — and it teaches them in the company of the discipline that decides whether any of them are worth deploying: splitting, validation, calibration, cost-sensitive evaluation and monitoring. The methods are the easy part. Every library will run them for you. The discipline is what distinguishes an engineer from someone who can call a function.

The order of learning

The chapters follow the order in which understanding arrives, not the order of a syllabus.

We begin with the shape of a learning problem: what it means to learn from examples, and what you are assuming when you do. Then data, because a model is a claim about data and the claim is only as good as the sample. Then error, because the choice of what to minimise decides what the model becomes.

The middle of the volume builds four methods that are simple enough to compute by hand and useful enough to deploy: linear regression, logistic regression, nearest neighbours, and trees with their ensembles. Each is introduced with numbers, worked through on a small dataset, and examined for the way it fails. A reader who can fit a line by hand, work a logistic model at a single point, and split a small table by hand has the intuition that later volumes will formalise.

The last part is the part that gets skipped in courses and costs money in practice: feature pipelines, validation under leakage, metrics that respect costs, deployment, drift, retraining, and the question of whether machine learning is the right tool at all.

What you need

A pencil, paper, and a calculator. Python helps from Chapter 4 onwards, and the examples are written so that they can be followed by typing them in rather than by understanding a framework. No mathematics beyond arithmetic, percentages and the idea of a graph is assumed. Where a formula appears, it is followed by a sentence saying what it does, and usually by a worked number.

A warning that is also a promise

Machine learning is the part of artificial intelligence where it is easiest to fool yourself and hardest to notice. A model can be beautiful, fast, accurate on its test set and useless in the world, and no error message will appear. The promise of this volume is that you will learn to see the difference before the deployment, rather than after it.

How This Volume Is Arranged

Part One establishes the vocabulary: what a learning problem is, where data comes from, and what it means to be wrong. It contains no algorithms, and it is the part to reread when a project starts behaving strangely.

Part Two builds four methods in order of increasing flexibility: a straight line, a probability, a neighbourhood, and a tree. Each chapter fits the method on a small table of numbers, examines what it can and cannot express, and names the way it fails.

Part Three is the discipline. It covers overfitting as a trade between two kinds of error, validation as a procedure that can be done correctly or incorrectly, and evaluation as a decision about costs rather than a single number.

Part Four takes the model out of the notebook. It covers the pipeline that turns raw columns into predictions, the deployment that keeps a model alive, and the cases where the honest answer is that machine learning is the wrong tool.

Every chapter ends with work: calculations to do by hand, small experiments on data you can invent, and one question that has no single right answer. The calculations matter more than the reading, because the failure modes of this subject are numerical and only practice makes them visible.

PART ONE

The Shape of Learning

CHAPTER ONE

The Shape of a Learning Problem

Two tasks sit next to each other in the same company. The first is deciding whether an incoming email is spam. The second is computing the monthly payroll. Both are done by software; both involve many rules; only one of them should be learned from data.

The difference is not difficulty. Payroll is complicated and tedious, and it is written down in law and in contracts. Every rule can be stated, checked and audited by an accountant. Spam has no such document. Nobody can write the rule that separates a legitimate marketing email from a fraudulent one, because the difference depends on context, language, sender history and a thousand small signals that people recognise but cannot articulate.

That gap — between knowledge that can be written down and knowledge that can only be demonstrated — is where machine learning lives. Everything in this volume follows from taking it seriously.

Three Kinds of Learning

Learning problems come in three shapes, and the shape decides what you can measure.

In **supervised learning** you have examples with answers. A photograph labelled "cat", a transaction labelled "fraud", a house with a known price. The task is to produce the answer for a new example. If the answer is a category, the task is **classification**. If it is a number, the task is **regression**.

In **unsupervised learning** you have examples without answers. The task is to find structure: groups of similar customers, unusual sensor readings, a smaller set of numbers that captures most of the variation in a table. There is no right answer to compare against, which makes evaluation much harder and makes this the part of the field most often overclaimed.

In **reinforcement learning** you have neither answers nor a fixed dataset. An agent acts, the world responds, and a reward arrives. The task is to learn a policy that collects rewards over time. That is the subject of its own volume, and it is mentioned here only so that the three shapes can be seen together.

Shape	You have	You produce	Typical task
Supervised, classification	Examples with categories	A category	Spam, defects, diagnosis
Supervised, regression	Examples with numbers	A number	Demand, temperature, price
Unsupervised	Examples only	Structure	Segments, anomalies, compression
Reinforcement	Actions and rewards	A policy	Control, games, routing

A Model Is a Guessing Machine With Dials

Strip away the vocabulary and a model is a function with adjustable numbers in it.

A straight line has two dials: a slope and an intercept. A decision tree has a set of questions and thresholds. A neural network has millions of dials. Learning is the process of turning the dials until the function's outputs match the answers you have, as closely as possible, on the examples you have.

The set of functions a model can represent is called its **hypothesis space**, and choosing it is the most consequential decision in the project, because it decides what the model can ever learn.

A straight line can represent a relationship that rises steadily. It cannot represent a relationship that rises, falls and rises again, or one that depends on the interaction of two inputs. A tree can represent sharp thresholds and interactions. It struggles to represent a smooth curve. A neural network can represent almost anything, which is precisely why it can also memorise noise.

The preference for one kind of function over another is called **inductive bias**, and it is not a defect. It is the reason learning works at all. If a model had no bias, every function consistent with the data would be equally acceptable, including the absurd ones, and no amount of data would single out the sensible one. Choosing a bias that matches the world is the craft.

The Bet at the Centre of It

Here is the uncomfortable heart of the subject. You fit the model to the examples you have, and then you use it on examples you do not have.

The fit is measurable and the future is not. Everything that follows — splitting, validation, regularisation, monitoring — exists to manage that gap and to keep it honest.

Two failures lie on either side.

If the model is too simple for the pattern, it will be wrong on the training data and wrong on new data. It has **underfit**. A straight line through a curve is underfit: it misses the shape and no amount of extra data will fix the choice of a line.

If the model is too flexible, it will fit the training data beautifully and be wrong on new data, because it has learned the particular noise of the sample rather than the pattern underneath. It has **overfit**. This failure is more insidious because the training number looks excellent.

The skill is to sit between the two, and the only instrument that tells you where you are is a set of data the model has never seen. That is why the discipline in Part Three exists.

Framing: Three Questions Before Any Code

Most failed projects were framed badly rather than modelled badly. Three questions, asked in order, prevent most of it.

What exactly am I predicting? "Predict failure" is not a task. "Estimate the probability that this pump will fail within the next 30 days" is. The first has no unit, no horizon and no way to be scored. The second does.

What will I know at the moment of prediction? Every column in the table must be available then. A model that uses the repair date to predict repairs is a model that has read the future, and it will score beautifully and deploy uselessly.

What does a mistake cost? A missed failure and a false alarm are not the same. Once the costs are written down, the choice of threshold, metric and even model follows from them.

Vague framing	Usable framing
Predict machine failure	Probability of failure within 30 days, from vibration and temperature, for machines under 5 years old
Improve customer retention	Probability that a customer cancels within 60 days, from usage and support history
Detect anomalies in production	Flag parts whose surface image differs from the reference population, reviewed by an inspector within 4 hours

The same underlying problem can be framed several ways, and each framing produces a different dataset and a different metric.

Predicting a pump's failure can be **classification** — will it fail in the next month, yes or no? It can be **regression** — how many days until it fails? Or it can be **unsupervised** — is this vibration pattern unusual compared with the fleet? The classification framing needs labelled failures, of which there may be only four in a year. The regression framing needs even more precise labels. The unsupervised framing needs no labels at all, and it will find every unusual pump, including the ones that are unusual because they are new.

None of those is wrong. They answer slightly different questions, and the choice depends on what data exists and what a decision will be made with the answer.

What Machine Learning Is Not

Two expectations are worth removing before the methods arrive.

It is not a way to avoid understanding the problem. A model can find a pattern in a table without anybody knowing why the pattern exists, and that is fine until the pattern changes, at which point nobody can repair it. The teams that do this well keep a human explanation beside the statistical one.

It is not automatically better than a rule. A rule written by a domain expert is auditable, explainable and free to run. Machine learning earns its place when the pattern is real, large and too complex to write down, and when there is data to prove it. Chapter 13 takes this seriously, and it is one of the most useful chapters in the volume.

How Much Data Is Enough?

This is the question every project asks first, and the honest answer is a curve rather than a number.

A **learning curve** plots performance against the number of training examples. In most projects it rises steeply for a while, then flattens. The flat part is the signal: adding data there buys almost nothing, and the remaining error is a mixture of irreducible noise and model bias. The steep part is where data is worth collecting, and where a small investment pays.

Two rules of thumb are useful as starting points, not as laws.

For a linear model with a modest number of features, ten to twenty examples per parameter is a common minimum. A model with five coefficients wants at least fifty to a hundred examples, and more if the data is noisy.

For classification, what matters is the number of examples of the rarer class. A dataset with 10,000 rows and 40 positives is a dataset of 40 examples for the purpose of learning what a positive looks like. Flexible models need several hundred positives before their advantage over a simple model shows up.

There is a third consideration that no rule of thumb captures: the number of *distinct situations* the model must handle. A welding dataset from one fixture may need a few hundred images. The same model, applied to twenty fixtures, needs examples from all twenty, because the variation between fixtures is the thing that will break it.

The practical procedure is cheap. Fit your candidate model on 10%, 25%, 50% and 100% of the training data, plot validation performance, and look at the shape. If the curve is still climbing at 100%, collect more data. If it has flattened, collect better features, or accept the floor and move on to deployment.

Training fraction	Validation score	Interpretation
10%	0.61	Steep learning
25%	0.72	Still climbing
50%	0.79	Approaching the plateau
100%	0.81	Flattening; data is no longer the constraint

Problems

1. For each task, say whether it is classification, regression, unsupervised or not a learning problem at all, and why: sorting mail by postcode; predicting tomorrow's electricity demand; grouping customers by behaviour; deciding whether a photographed weld has a crack; computing interest on a loan; detecting an unusual heartbeat in a patient's recording.

2. Rewrite these three wishes as usable framings with a target, a horizon and a measure: "use AI to reduce downtime", "make our hiring fairer", "predict which students will fail".

3. A model has two dials and is fitted to six points that lie almost on a straight line with a little scatter. Describe what happens if the model has two dials, and what happens if it has six.

4. Name one decision in your own life where you use a rule, and one where you use judgement learned from experience. What makes the first rule-shaped and the second example-shaped?

CHAPTER TWO

Data: Where It Comes From and What It Costs

A team builds a system that finds cracks in welds. They collect twelve thousand images from one factory, train a model, and reach 98% accuracy. The model is installed on a second production line in another country, where the lighting is different, the camera is a different model, and the welds are made by a different process. Accuracy falls to 61%, and nobody can explain why, because nobody wrote down where the twelve thousand images came from.

Data is the largest cost, the largest source of failure, and the part of machine learning that is least often taught. This chapter is about getting it, understanding what it represents, and being honest about what it does not.

Four Sources, Four Biases

Data arrives in four ways, and each carries its own distortion.

Sensors and logs record what happened. They are cheap, plentiful and mercilessly honest about one thing only: the conditions under which they were installed. A temperature dataset from a factory in one climate is a dataset about that climate.

Human labelling attaches answers to examples. It is expensive, inconsistent and indispensable for supervised learning. Two experts often disagree, and the disagreement is itself information: a task where experts agree only 85% of the time cannot support a model judged against a single label.

Transactions record decisions that people made: loans approved, candidates hired, claims paid. They are convenient and deeply biased, because they record the outcome of a process that was already selective. A model trained on approved loans learns what the approvers liked, not who would have repaid.

Purchase buys you a dataset someone else collected for a different purpose. It is fast and it is usually a poor fit, because the collection decisions were driven by someone else's problem.

The common thread is that a dataset is a record of how it was collected, and the model learns the collection as faithfully as it learns the phenomenon.

Coverage, and the Sample Is Not the World

The assumption underneath every model is that the future resembles the sample. When that assumption fails, everything else is decoration.

Four kinds of coverage failure matter most.

Population coverage. A hospital's model is trained on its own patients. Another hospital serves a different population with different age structure, different languages and different referral patterns. The model's accuracy there is an open question, and the honest answer is "we do not know until we measure".

Condition coverage. Photographs taken in daylight; readings taken in summer; transactions from a stable economy. The conditions absent from the data are the conditions in which the model will eventually be asked to work.

Time coverage. A model trained on five years of history contains the past's relationships. When a competitor enters, a regulation changes or a pandemic arrives, the relationship moves and the model does not.

Rare-event coverage. Fraud, machine failure and serious medical events are rare by definition. A dataset with forty examples of the thing you care about cannot support a model with a hundred parameters, whatever the total row count says.

A useful habit is to write a short **dataset description** before modelling: who or what is in it, where it came from, when, under what conditions, and what is deliberately missing. Google's "Datasheets for Datasets" paper formalised this, and a page of it prevents most surprises.

Labels Cost More Than You Think

Labelling is usually the dominant cost, and the arithmetic is sobering.

Suppose a weld image takes a trained inspector twenty seconds to label: open, zoom, decide, enter. One thousand images take 20,000 seconds, which is five and a half hours of uninterrupted work. In practice, with context switching and quality checks, plan for eight. For a model that needs twenty thousand labelled examples, that is four months of one person's time, before any review.

The cost changes the design. It pushes teams towards:

Active learning, where the model chooses the examples it is least sure about and a human labels only those, which can cut the labelling requirement by a factor of three or four;

Weak supervision, where cheaper signals — a maintenance record that says "replaced bearing" — stand in for expensive labels, at the cost of noise;

Synthetic data, generated from a simulation or a model, which is cheap and only as good as the simulation's fidelity.

Each of these trades label cost for some other risk, and naming the trade is part of the engineering.

Cleaning, and the Errors That Survive

Real tables are dirty in predictable ways.

Missing values are rarely random. A sensor that fails when hot produces missing readings precisely when the machine is under stress, so deleting those rows deletes the hard cases and flatters the model. Decide explicitly: delete, impute with a median, or add a "was missing" indicator.

Duplicates appear when the same record is entered twice or the same image is stored under two names. They inflate the apparent dataset size and, worse, can put the same example in training and test. De-duplicate before splitting.

Unit and scale errors are the quiet killer. A temperature column with ninety-nine values in Celsius and one in Fahrenheit; a distance in millimetres among distances in metres. A single such row can dominate a squared-error loss and distort a model without producing an error message. Plot every column before you trust it.

Outliers are either errors or the most interesting rows in the table. Deciding which is a domain question, not a statistical one: a pump vibrating at ten times the fleet average is either a broken sensor or a machine about to fail, and only someone who knows pumps can tell you.

Splitting Is a Design Decision

The train–validation–test split is usually taught as a random 70/20/10. That is one choice among several, and often the wrong one.

Split	Use when	Why
Random	Examples are independent and exchangeable	Standard baseline
Grouped	Several rows belong to one person, machine or image	Prevents the model learning the group
Temporal	The model will predict the future	Mimics deployment, prevents time leakage
Stratified	Classes are imbalanced	Keeps the rare class represented in each part
By site	The model will be used in another location	Measures transfer honestly

A medical dataset with several X-rays per patient must be split by patient. A demand model must be split by date, with all training data before all test data. A model destined for another factory should be tested on that factory, or on held-out sites if the data exists.

None of these splits is more sophisticated than the others. Each answers a different question about the world the model will meet, and the right question is the one the deployment will ask.

Problems

1. You have a dataset of 50,000 support tickets labelled by category, collected over three years. Name three coverage failures you would expect and the effect each would have.

2. Estimate the labelling cost for 8,000 images at 15 seconds each, with a 20% overhead for quality review. How would the number change if active learning cut the requirement by 60%?

3. A colleague deletes all rows with missing sensor values, reducing the dataset from 20,000 to 14,000 rows and improving accuracy from 81% to 88%. Explain what has probably happened and what you would check.

4. You must split a dataset of loan applications from 2015 to 2024. Describe the split you would use if the model is to be deployed next year, and why a random split would mislead you.

5. Write a one-page description of a dataset you can access, real or invented: who is in it, where it came from, and what is missing.

CHAPTER THREE

Measuring Error

A model is trained by being punished. The punishment is a single number, the **loss**, and the model adjusts its dials to make that number smaller. Everything about what the model becomes follows from the shape of that number.

This is a short chapter with a long reach. Two models trained on identical data with identical architecture will behave differently if they are trained with different losses, because the loss is where you state what you consider a mistake.

Squared Error, and What It Rewards

For regression, the default loss is the squared difference between prediction and truth, averaged over the data.

$$\text{loss} = \text{average of } (\text{prediction} - \text{truth})^2$$

Squaring has two effects, and both matter. It removes the sign, so that misses above and below do not cancel. And it punishes large misses disproportionately: an error of 0.3 costs nine times as much as an error of 0.1.

A model trained this way becomes careful about extremes. If one row in the table is a typo of ten times the true value, the model will deform itself to reduce that one error, often at the cost of being worse everywhere else. That is why cleaning outliers matters before fitting, and why mean absolute error is the alternative when the tails are noise.

For reporting, the root mean squared error is preferred because it returns to the original units: a prediction of machine temperature in degrees should be scored in degrees, not in degrees squared. The mean absolute error is preferred when you want to describe the typical mistake rather than the worst one.

Cross-Entropy, and Why It Is Not Squared Error

For classification, the model usually outputs a probability, and the loss is different.

If the model says the probability of the true class is p, the cross-entropy loss for that example is

```
loss = -log(p)
```

Take three predictions for a true class:

- p = 0.9 gives a loss of 0.105.
- p = 0.5 gives a loss of 0.693.
- p = 0.1 gives a loss of 2.303.

Two things are worth noticing. Confident and right is cheap. Confident and wrong is expensive — the loss grows without limit as p approaches zero — which is exactly the behaviour you want from a system that will be used to make decisions.

Compare this with squared error on the same probabilities: 0.01, 0.25 and 0.81. Squared error punishes a confident mistake less severely than cross-entropy does, and it punishes a correct confident answer almost not at all. For probabilities, cross-entropy is the loss that matches the question.

Prediction for the true class	Squared error	Cross-entropy
0.9	0.01	0.11
0.7	0.09	0.36
0.5	0.25	0.69
0.1	0.81	2.30

R-Squared and the Temptation of a Single Number

For regression, software often reports **R^2**, the fraction of the variation in the target that the model explains. It ranges from zero (no better than predicting the average) to one (perfect), and it can be negative when the model is worse than the average.

The appeal is obvious: one number, bounded, comparable. The danger is equally obvious once you say it out loud. R^2 says nothing about the size of the

errors in the original units, nothing about the worst case, and nothing about whether the model's assumptions hold outside the range of the data.

A model can have an R^2 of 0.99 on a dataset spanning 10 to 30 degrees and be useless at 40, because it has never seen 40 and its straight line may not bend the way physics does. Always report R^2 beside the root mean squared error, and always plot the residuals.

Baselines: The Number You Must Beat

Before any model, compute the simplest possible predictor. It takes ten minutes and it redefines the whole project.

For regression, the baseline is usually the mean of the target. A model that cannot beat "predict the average" has learned nothing.

For classification, the baseline is the majority class. If 96% of parts are good, a model that says "good" to everything scores 96% accuracy, and any reported accuracy must be read against that.

For forecasting, the baseline is "tomorrow will be like today", which is surprisingly hard to beat for many real series.

Problem	Baseline	Typical baseline score
Will this part be defective? (4% defects)	Always "not defective"	96% accuracy
What will demand be tomorrow?	Today's value	Often 80–90% of the best model's performance
Which customers will churn? (8% churn)	Always "stay"	92% accuracy

A result is a comparison. "Our model achieved 91% accuracy" is not a finding; "our model achieved 91% against a 96% baseline" is a finding, and it is a bad one. The habit of computing the baseline first, and reporting it beside every result, is the single cheapest protection against self-deception in this field.

Choosing the Loss Before the Model

The practical sequence is: decide what a mistake is, express it as a number, then choose a model that can minimise it.

A maintenance team cares about the cost of an unexpected failure and the cost of an unnecessary inspection. Those two costs imply a loss that weights the

two errors differently — a cost matrix, which Chapter 10 turns into thresholds. A team training with plain accuracy has decided, silently, that those two costs are equal, and that decision was made by nobody.

The same logic applies to a regression. If over-predicting a drug dose is dangerous and under-predicting is merely ineffective, symmetric squared error is the wrong loss, because it treats a 5 mg overdose and a 5 mg underdose as equally bad. The fix is an asymmetric loss, and it is a design decision that belongs to the clinicians.

The Accuracy Trap

Accuracy is the ratio of correct predictions to all predictions, and it has a property that makes it dangerous: it is dominated by the majority class.

Take a factory producing 1,000 parts, 30 of which are defective. A system that declares every part good scores (970/1000) = 97% accuracy, and catches nothing. A real system that catches 24 of the 30 defects, at the cost of flagging 36 good parts, scores (24 + 934)/1000 = 95.8% accuracy — lower than doing nothing, and vastly more useful.

System	True positives	False positives	Missed defects	Accuracy
Flag nothing	0	0	30	97.0%
Real detector	24	36	6	95.8%

The table is the whole argument for cost-sensitive evaluation, and it can be summarised in one sentence: when the thing you care about is rare, accuracy rewards ignoring it.

Three habits follow.

Always compute the majority-class baseline first, and report it beside every accuracy figure.

Report precision and recall, or a confusion matrix, so the reader can see what the model actually finds.

Keep the class proportions of your test set the same as production. A model evaluated on a deliberately balanced test set — half positives, half negatives — will look far better than it will perform in a world where positives are 3%.

The last point deserves an example, because balanced test sets are common in research and rare in life. A detector with 80% recall and 40% precision, eval-

uated on a balanced set, looks like a reasonable system. Evaluated at 3% prevalence, the same sensitivity and specificity produce a precision of about 5%, because the false positives overwhelm the true ones. Neither number is a lie. The second is the one that describes the factory.

Problems

1. Compute the squared error and the cross-entropy for predictions of 0.95, 0.8, 0.6, 0.4 and 0.05 on the true class. Plot both on the same axes by hand and describe the difference in shape.

2. A regression model has an R^2 of 0.94 and an RMSE of 0.8 °C. A second has an R^2 of 0.90 and an RMSE of 0.5 °C. Which would you deploy for temperature control, and why?

3. A dataset has 3% defects. Compute the accuracy of the majority-class baseline. Then explain why a model with 94% accuracy might be worse than useless.

4. Design an asymmetric loss for a system that predicts how much insulin to deliver, where a dose 1 unit too high is four times as costly as a dose 1 unit too low. Write the loss as a formula and explain how it changes the model's behaviour.

5. Take a reported result you have seen in the news — accuracy, precision, error rate — and write the baseline that should have accompanied it.

PART TWO

Four Methods Worth Knowing

CHAPTER FOUR

A Straight Line, Fitted Properly

A bearing runs for five hours. Its housing warms from 12 °C above ambient to 21 °C. Plot the two against each other and the points fall almost on a line. If that relationship holds, the temperature tells you how long the machine has been working, and — more usefully — a temperature that rises faster than the line predicts tells you something is wrong.

The straight line is the oldest model in statistics and the most useful first model in machine learning. It is not a toy, and it is not a stepping stone to something better. Many real relationships are close enough to linear that the line wins on every axis that matters: it is cheap, it is explainable, it needs little data, and it fails in ways you can see.

The Line and Its Two Numbers

A straight line is written

```
prediction = intercept + slope x input
```

The **slope** says how much the output changes when the input increases by one unit. The **intercept** says what the output is when the input is zero. Both have units, and the units are the fastest way to check that a fit is sensible.

Here is a table of five observations from a test rig: hours run, and temperature rise above ambient.

Hours run	Temperature rise (°C)
1	12.1
2	13.9
3	17.2
4	18.8
5	21.4

Fitting by eye, the line should rise by about 2.3 degrees per hour and pass near 16.7 at three hours. The arithmetic gives the same answer and tells you how confident to be.

Least Squares, by Hand

The standard method chooses the line that minimises the sum of squared vertical distances from the points to the line. The formulas are short, and computing them once by hand removes the mystery.

First, the averages: the mean of the hours is 3, and the mean of the temperatures is 16.68.

Then two sums. For each point, take the deviation in hours from the mean and the deviation in temperature from the mean, multiply them, and add:

(-2)(-4.58) + (-1)(-2.78) + (0)(0.52) + (1)(2.12) + (2)(4.72) = 23.5

Then take the squared deviations in hours and add:

4 + 1 + 0 + 1 + 4 = 10

The slope is 23.5 / 10 = 2.35 degrees per hour. The intercept follows from the line passing through the average point: 16.68 - 2.35 x 3 = 9.63 degrees.

So the model is

temperature rise = 9.63 + 2.35 x hours

Two sanity checks, both done in seconds. The units are degrees per hour, which is what a slope should be. And the line passes through the average of the data, which it must, because least squares always does.

Point	Hours	Actual	Predicted	Residual
1	1	12.1	11.98	+0.12
2	2	13.9	14.33	-0.43
3	3	17.2	16.68	+0.52
4	4	18.8	19.03	-0.23
5	5	21.4	21.38	+0.02

Reading What You Fitted

The residuals — the differences between what happened and what the line predicted — are the most informative part of the output, and they are the part most often ignored.

The sum of squared residuals here is 0.523. The total variation of the temperatures around their mean is 55.75. The ratio gives $R^2 = 1 - 0.523/55.75 = 0.991$, which means the line explains 99% of the variation. The root mean squared error is 0.32 °C, and the mean absolute error is 0.26 °C.

Those four numbers describe different things. R^2 says the shape is right. RMSE says the typical error in degrees. The residual table says whether the errors are patternless. Here they alternate sign, which is what you want: no obvious curve left in the data.

The residuals also reveal what the model cannot tell you. Every point has a positive residual in the middle and negative at the ends — mild curvature. A real bearing's temperature does not rise linearly forever: it approaches an equilibrium where heat in equals heat out. The line is a good local description and a poor global one, and the residuals are the evidence.

Where a Straight Line Breaks

Four failures are worth memorising, because each has a recognisable signature.

Non-linearity. The relationship curves. The residual plot shows a systematic bow: positive in the middle, negative at the ends, or the reverse. The fix is to add a squared term, transform an input, or fit a more flexible model.

Extrapolation. The line continues beyond the data because a line has no way to stop. A model fitted between 1 and 5 hours will happily predict 60 degrees at 20 hours, and be wrong, because the physics saturates. Never trust a regression outside the range of its inputs without independent reason.

Interaction. The effect of one input depends on another. Temperature rise may depend on hours *and* on ambient humidity, and the dependence may be different at high humidity. A model with additive terms cannot represent that; it needs a product term or a different method.

Changing variance. The errors grow with the input: small at one hour, large at five. Least squares assumes they do not, so the fit is dominated by the noisiest region. A log transform or weighted fitting is the usual repair.

Symptom	Likely cause	First thing to try
Bowed residual pattern	Non-linearity	Add a squared term
Errors growing with input	Changing variance	Fit on a log scale
Line misses a subgroup	Missing variable	Add the group or an interaction
Excellent fit, terrible predictions at new values	Extrapolation	Restrict the range, or use physics

More Than One Input

Most real relationships involve several inputs, and the extension is straightforward: the model becomes

prediction = intercept + slope■ x input■ + slope■ x input■ + …

Each slope now means something slightly different from before. It is the change in the output when that input rises by one unit, **holding the others constant**. That phrase is doing a lot of work, and it is the reason regression coefficients are so often misinterpreted.

Suppose a model of temperature rise is fitted with two inputs: hours run and ambient humidity, giving

rise = 9.4 + 2.3 x hours + 0.12 x humidity

The reading is: each additional hour adds 2.3 degrees, for machines running in the same humidity; each percentage point of humidity adds 0.12 degrees, for machines that have both run the same number of hours. The second clause is almost never what a manager wants to know, because in practice humidity and hours are correlated.

That correlation produces the most common pathology in multiple regression, called **collinearity**. When two inputs move together, the model cannot tell which one is responsible, and the coefficients become unstable: add a few rows and the humidity coefficient swings from +0.12 to -0.30 while the predictions barely change. The model still predicts; it has simply stopped being a description.

The diagnostic is to compute the correlation between every pair of inputs. Correlations above about 0.8 are a warning. The fixes are to drop one of the

pair, to combine them into a single feature, or to use a regularised model that distributes the effect stably rather than assigning it arbitrarily.

Symptom	Likely cause	Response
Coefficients flip sign between datasets	Collinearity	Drop or combine a feature
Huge coefficient with tiny effect	Scaling or collinearity	Standardise inputs
Good predictions, nonsensical coefficients	Confounding	Use for prediction only

This is the point at which prediction and explanation part company. A model can be excellent at predicting while its coefficients are meaningless as descriptions, and a model whose coefficients are meaningful may predict poorly. Deciding which you need is a requirement, not a technicality.

Why Start Here

A more flexible model would fit these five points better. It would also be harder to check, harder to explain, and no more accurate on the region that matters. Starting with a line has three practical benefits.

It gives you a baseline that is not a flat average, so later models must beat something reasonable.

It exposes the structure of the data. Fitting a line tells you the sign, size and units of each effect, which is often the actual answer the business wanted.

It fails visibly. When a straight line is wrong, the residual plot says so. When a large model is wrong, the loss is lower and the failure is hidden.

Problems

1. Fit a line by hand to the five points above. Confirm the slope and intercept, then predict the temperature at 6 hours and at 0.5 hours. Which prediction do you trust less, and why?

2. A regression predicts house price from floor area and gives a slope of 850 per square metre and an intercept of 40,000. Interpret both numbers in words, and say which of them is physically meaningful.

3. The residual plot of a model shows a clear U shape. What does that tell you, and what would you do next?

4. A model of energy use versus production volume is fitted on data from 20% to 90% utilisation. A manager asks for a prediction at 110%. Write the sentence you would say, and the number you would refuse to give.

5. Explain in three sentences why a straight line can be the best model even when a curved one fits the training data better.

CHAPTER FIVE

From a Number to a Probability

The previous chapter predicted a temperature. This one predicts whether a part will fail, a customer will leave, or a patient has a condition. The output is a category, and the natural way to produce a category is to produce a probability and then apply a threshold.

That move — from score to probability to decision — is where most of the practical difficulty of classification lives. A model that outputs 0.73 has not said "yes". It has said something that a human, a policy or a cost calculation must convert into an action.

Why Not Just Use a Line?

You could fit a straight line to a yes/no outcome, treating "yes" as 1 and "no" as 0, and many people have. It produces predictions like 1.4 and -0.3, which are not probabilities, and it treats a case near the boundary as no more uncertain than a case far from it.

The better approach keeps the linear part but passes it through a curve that squeezes any number into the range between 0 and 1. The curve is the **logistic function**:

```
probability = 1 / (1 + e^(-z))
```

where z is the linear score, built from the inputs exactly as in regression. Large positive z gives a probability near 1. Large negative z gives a probability near 0. z equal to zero gives exactly 0.5.

The quantity z is called the **log-odds**, and it has a clean interpretation: it is the logarithm of the ratio between the probability of the event and the probability of no event. A log-odds of zero means even chances. A log-odds of 1 means about 2.7 to 1 in favour.

Linear score z	Probability	Odds
-3	0.05	1 to 19 against
-2	0.12	1 to 7.4 against
-1	0.27	1 to 2.7 against
0	0.50	Even
1	0.73	2.7 to 1 in favour
2	0.88	7.4 to 1 in favour
3	0.95	19 to 1 in favour

The table is worth internalising, because it tells you how a logistic model behaves: most of the action happens in a narrow band around zero, and the tails saturate. A large change in a feature that is already pushing the score to 3 will barely move the probability, which is why interpreting coefficients as "effect sizes" needs care.

A Small Model, Worked

Suppose a maintenance model uses two inputs: vibration in mm/s and temperature in °C. Fit the parameters, and suppose the result is

```
z = -8.0 + 1.4 x vibration + 0.06 x temperature
```

Take a pump reading 2.5 mm/s and 50 °C:

```
z = -8.0 + 3.5 + 3.0 = -1.5
```

The probability is 1 / (1 + e^1.5) = 1 / (1 + 4.48) = 0.18. The model says an 18% chance of failure within the month.

Now take a pump at 3.5 mm/s and 55 °C:

```
z = -8.0 + 4.9 + 3.3 = 0.2
```

The probability is 1 / (1 + e^-0.2) = 0.55. A one-unit change in vibration and a five-degree change in temperature moved the probability from 18% to 55%.

This sensitivity is a property of the model, not of the pumps, and it is a warning. If the true relationship were gentler, the fitted model would still produce a dramatic swing around the boundary, and decisions built on it would be brittle. The coefficients are only as good as the data, and the data is often thirty

examples.

How the Parameters Are Found

Linear regression has a formula: the least-squares solution can be written down in one line. Logistic regression does not, and the parameters are found by search.

The loss is cross-entropy, which for a single example with true label y and predicted probability p is

```
loss = -log(p) if y is 1, and -log(1 - p) if y is 0
```

The training procedure starts with arbitrary parameters, computes the loss, works out which direction to nudge each parameter to reduce it, and repeats. That procedure is gradient descent, and its details belong to the volume on neural networks. What matters here is that the loss is smooth and the problem is well behaved, so the search usually finds a good answer — and that the answer can be unstable when features are highly correlated or when one class is very rare.

The Threshold Is a Decision, Not a Default

Software will report a 0.5 threshold because 0.5 is the obvious place to split a probability. In most real systems it is the wrong place.

The correct threshold comes from the relative costs. If a missed failure costs 500 and a false alarm costs 20, you should flag far more cases than the 0.5 rule would. Chapter 10 turns that intuition into arithmetic; here is the short version.

Suppose the model's probability is a genuine probability. Then flagging when p is above a threshold t costs

```
expected cost = (1 - p) x cost of false alarm + p x cost of miss
```

Setting the two costs at 20 and 500, the threshold at which the expected costs of acting and not acting are equal is 20 / (20 + 500), about 0.038. Anything above a 3.8% chance of failure is worth inspecting, because the inspection is cheap and the miss is not.

Miss cost	False alarm cost	Threshold
500	20	0.04
100	20	0.17
20	20	0.50
20	100	0.83

The table explains a great deal of real-world behaviour. Screening programmes use low thresholds because missing a disease is expensive. Spam filters use higher thresholds because blocking a real message annoys users more than letting one spam through. Neither is a property of the model; both are properties of the decision.

What a Coefficient Means, and What It Does Not

Because logistic regression is linear in the log-odds, a coefficient has a clean reading: increasing that feature by one unit adds its coefficient to the log-odds, which multiplies the odds by e raised to that coefficient.

A coefficient of 1.4 multiplies the odds by about 4. A coefficient of 0.06 multiplies them by 1.06, or 6%.

Two cautions. The effect on the probability depends on where you start: the same coefficient produces a large change in probability near 0.5 and a tiny one near 0.95. And a coefficient is not a cause. It describes an association in this dataset, and datasets encode the processes that produced them, including the biases of the people who made the decisions recorded in them.

Problems

1. Compute the probability for z = 0, 1, 2, 3 using the logistic function, and check your answers against the table.

2. A model has z = -6 + 0.9 x vibration + 0.05 x temperature. Compute the probability for a pump at 3 mm/s and 60 °C, and for one at 5 mm/s and 40 °C. Which feature is doing more work, and how do you know?

3. With a miss cost of 400 and a false alarm cost of 25, compute the threshold. Then explain in two sentences what a manager would notice if you deployed at 0.5 instead.

4. A coefficient multiplies the odds by 3. What does that mean in words, and why is it not the same as tripling the probability?

5. Write the logistic model you would use to decide whether to send a customer a retention offer. Name the inputs, the label, the horizon, the costs and the threshold — before writing any code.

CHAPTER SIX

Neighbours, and the Trouble With Many Dimensions

The simplest classifier in the world stores every example it has ever seen. When a new case arrives, it finds the most similar stored cases and copies their answer. There is no training, no parameters, no optimisation. It is a filing cabinet with a search function, and on some problems it is startlingly hard to beat.

The method is called **k-nearest neighbours**, or kNN, and it is worth understanding for two reasons. It shows what learning looks like when the model has no opinion at all, and its failure in high dimensions explains why so much of modern machine learning looks the way it does.

Distance Is a Choice You Make

"Most similar" is doing a lot of work in that description, and it is decided by a distance measure.

The usual choice is straight-line distance: for two examples with features x and y, square the differences in each feature, add them, and take the square root. With two features, it is Pythagoras.

The choice of measure is not neutral. If one feature is measured in millimetres and another in kilometres, the first will dominate every comparison. So features are usually **scaled** first: subtract the mean and divide by the spread, so that each feature contributes on the same footing.

Take two pumps. Pump A: vibration 2.0 mm/s, temperature 48 °C. Pump B: vibration 3.0 mm/s, temperature 49 °C. Pump C: vibration 2.1 mm/s, temperature 60 °C.

Without scaling, the raw squared distances are 1 + 1 = 2 to B, and 0.01 + 144 = 144 to C. B is nearest, which matches intuition. Now suppose temperature were recorded in tenths of a degree, so that C is 120 units away rather than 12. The distance to C becomes 0.01 + 14,400, and the distance to B becomes 1 + 100. B is still nearest, but the temperature now dominates so completely that

vibration has stopped mattering. A pump with wildly different vibration but a similar temperature would be judged close.

Scaling removes that artefact. After standardisation, each feature's differences are measured in units of its own spread, and the comparison becomes about the data rather than about the choice of sensor.

Choosing k

With k equal to one, the classifier copies the single nearest example. That is maximally flexible and maximally fragile: a single mislabelled point creates a small island of wrong answers around itself.

With large k, the classifier averages over a wide neighbourhood and eventually predicts the majority class everywhere. That is stable and useless.

The useful range is in between, and the choice is the familiar trade. A small k gives a model with low bias and high variance: it can represent complicated boundaries and it changes a lot when the data changes. A large k gives high bias and low variance: smooth, stable and unable to capture detail.

k	Boundary	Behaviour	Typical problem
1	Very jagged	Memorises every point	Fooled by label noise
3–5	Locally detailed	Good for clean data	Sensitive near class overlap
15–30	Smooth	Stable	Blurs genuine structure
Very large	Nearly flat	Predicts the majority	Useless

A rough starting point is the square root of the number of examples, so 10 for a hundred rows and 100 for ten thousand, but it is only a starting point. Ties matter too: with k even and two classes, a tie is possible, and the tie-break rule must be specified rather than left to the library.

The Curse of Dimensionality

Here is the fact that turned kNN from a curiosity into a cautionary tale.

In one dimension, a line segment's interior is most of its length; only the two ends are near the boundary. In two dimensions, a square's interior is still most of its area. But as dimensions rise, almost all of the volume of a shape ends up near its surface.

Take a cube of side 1 and shrink it by 10% on every side, so the inner cube has side 0.9. The inner cube's volume fraction is 0.9 to the power of the number of dimensions.

- In 2 dimensions, 0.81: about 19% of the volume is in the outer shell.
- In 10 dimensions, 0.35: 65% of the volume is in the shell.
- In 50 dimensions, 0.005: 99.5% of the volume is in the shell.

So in high dimensions, almost every point is an outlier, and "nearby" points are not near in any useful sense. Distances between random points concentrate: the nearest neighbour and the farthest neighbour become almost equally far away. The ratio of the two distances shrinks towards one as dimensions rise, which means the notion of a local neighbourhood dissolves.

The practical consequence is sample size. To cover a space evenly, the number of examples needed grows exponentially with the number of dimensions. Ten examples per axis in two dimensions is a hundred; in ten dimensions it is ten billion.

This is why feature selection and dimensionality reduction are not optional refinements. They are how you bring the data back into a range where local methods work, and it is why so much of modern practice is about learning a good low-dimensional representation before classifying.

Where Neighbours Win

For all that, kNN is not a museum piece. It is excellent in three situations.

Recommendation. "Customers who bought this also bought" is, underneath, a nearest-neighbour search over sparse vectors of purchases. The distances are strange and the data is enormous, but the principle is the same.

Imputation. To fill a missing value in a row, find the most similar complete rows and average their values. It respects local structure that a global mean destroys.

Sanity checks. A kNN model is a good first thing to run on a new tabular dataset, because if a flexible local method cannot beat a simple linear model, the data is probably not rich enough to justify anything complicated.

Its costs are worth naming: prediction requires comparing the new point with every stored point, so it is slow on large data, and it requires keeping the whole training set in memory. Modern libraries use trees and approximate

search to mitigate this, but the fundamental trade remains — kNN buys simplicity with storage and search time.

Problems

1. Standardise three features by hand: values 10, 12, 14 (mean 12, spread 2); 100, 110, 120 (mean 110, spread 10); and 1, 2, 3 (mean 2, spread 1). Show the standardised values and explain why the original scales are incomparable.

2. With five examples labelled 0, 0, 1, 1, 1, compute the kNN prediction for a new point equidistant from the two nearest examples, for k = 1, 3 and 5. What does the exercise demonstrate?

3. Compute 0.9 to the power of 5, 20 and 100. Interpret each as the fraction of a high-dimensional cube's volume that lies in its outer shell, and say what it means for the idea of a local neighbourhood.

4. A dataset has 200 rows and 60 features. Explain why kNN is likely to perform poorly, and name two things you would do before using it.

5. Describe a recommendation problem you use. What would "distance" mean in it, and which features would you have to scale before it made sense?

CHAPTER SEVEN

Trees, Forests, and the Art of Asking Questions

A maintenance engineer's diagnosis sounds like a tree. If the vibration is above 4 mm/s, look at the bearing. If the bearing is fine, check the alignment. If the alignment is fine and the temperature is above 60 °C, call the supplier. That structure — a sequence of yes-or-no questions, each chosen because it separates the cases well — is exactly what a decision tree learns.

Trees are the most useful family of models for tabular data, and they are the easiest to explain to somebody who does not work in machine learning. That combination, rather than any theoretical elegance, is why they dominate practical work on spreadsheets.

Splits and Impurity

A tree is built by choosing, at each step, the question that best separates the classes.

"Best" is measured by **impurity**: how mixed the group is. A group that is all one class has zero impurity. A group split evenly between two classes has maximum impurity for two classes, 0.5 by the Gini measure, or 1 bit by entropy.

Take ten welds, five cracked and five sound. The Gini impurity of the whole set is $1 - (0.5^2 + 0.5^2) = 0.5$.

Now try a split on vibration, above or below 3 mm/s. Suppose the low group has six welds, one cracked and five sound, and the high group has four, all cracked.

- The low group's impurity is $1 - ((1/6)^2 + (5/6)^2) = 0.278$.
- The high group's impurity is zero, because it is pure.
- The weighted impurity after the split is $(6/10)(0.278) + (4/10)(0) = 0.167$. The impurity fell from 0.5 to 0.167, a reduction of 0.333.

Now try a split on temperature instead, above or below 50 °C. Suppose each side has five welds, with two cracked on one side and three on the other. Each side's impurity is 0.48, and the weighted impurity is 0.48. The reduction is 0.02 — almost nothing.

The tree chooses vibration, and the arithmetic says why. It then repeats the procedure inside each branch, choosing the best remaining question, until the groups are pure or a stopping rule applies.

Split	Left group	Right group	Weighted impurity	Reduction
Vibration > 3	1 of 6	4 of 4	0.167	0.333
Temperature > 50	2 of 5	3 of 5	0.480	0.020

Entropy gives the same ordering by a different route. The whole set has one bit of uncertainty. After the vibration split, the low group carries 0.65 bits and the high group carries none, giving a weighted 0.39 and an information gain of 0.61 bits. Either measure works; both are greedy, and both look only one split ahead.

The Tree That Grows Too Far

Left alone, a tree will keep splitting until every training example sits in a leaf of its own. The training error goes to zero and the model becomes a very elaborate way of memorising the table.

Three stopping rules keep it honest, and each has a cost.

Maximum depth caps the number of questions. A depth of three to six is often enough for tabular data and produces a diagram a human can read.

Minimum samples per leaf stops a split that would leave fewer than, say, five examples in a leaf. It prevents the tree from building a rule around one unusual case.

Pruning grows the tree and then removes branches that do not improve validation performance. It is more expensive and usually better.

The failure mode of an overgrown tree is easy to describe and easy to miss: it predicts the training set perfectly, and its performance on new data collapses. The failure mode of an over-pruned tree is the opposite: it is stable, readable and too coarse to capture real structure. The validation set, again, is the only instrument that tells you where you are.

Why Forests Beat Trees

A single tree is unstable. Move ten rows in the dataset and the first split may change, which changes everything below it. That instability is a source of error, and it can be reduced by averaging.

A **random forest** builds hundreds of trees, each on a different random sample of the data, drawn with replacement; at each split, each tree is allowed to consider only a random subset of the features. The predictions are then averaged, or voted on.

The reason it works is worth stating precisely. Individual trees make errors, and those errors are partly independent, because the trees see different samples and different feature subsets. Averaging many partly independent errors reduces the variance of the average without increasing its bias much. The result is a model that is much more accurate than any of its members and much more stable than a single tree.

The costs are the usual ones. A forest of five hundred trees is five hundred times more expensive to build and to run, it is no longer readable as a diagram, and its predictions are harder to explain. In exchange it is often several points more accurate, which on a defect rate of 3% can be the difference between useful and decorative.

Model	Training cost	Prediction cost	Readable	Typical accuracy on tabular data
Single tree, depth 4	Low	Very low	Yes	Moderate
Single tree, unlimited	Low	Low	No, too large	Poor on new data
Random forest, 300 trees	High	High	No	Good
Boosted trees	High	Moderate	No	Usually best

Gradient boosting, the other dominant ensemble, builds trees one at a time, each correcting the errors of the previous ones. On tabular data it is usually the most accurate of the classical methods, at the cost of more careful tuning and a greater tendency to overfit if the number of trees is not controlled.

Feature Importance, and Its Lies

Trees offer a tempting number: **importance**, usually computed from how much each feature reduced impurity across the forest. It is easy to produce and easy to misread.

Impurity-based importance is biased towards features with many distinct values, such as continuous measurements or identifiers. A customer ID will often appear important because it can split the data many ways, not because it predicts anything.

Permutation importance is more honest: shuffle one column, measure how much the model's error increases, and repeat. If shuffling a feature destroys performance, it mattered. If it changes nothing, the feature was noise.

Neither measure tells you about causation, and both describe the model rather than the world. A feature can be important because it is a proxy for something else, and a feature can be unimportant because a correlated feature has taken its place.

Problems

1. Build the first split for a dataset of eight examples with four positives and four negatives. Compute the Gini impurity before and after a split that gives (4 positive, 0 negative) and (0 positive, 4 negative). What is the reduction?

2. Using the same dataset, construct a split that gives (2, 2) and (2, 2). Compute the reduction. Explain why a greedy tree would never choose it, even though it might lead to a better tree overall.

3. Explain in your own words why averaging many unstable models can produce a stable one. What has to be true of their errors?

4. A random forest reports that customer ID is the most important feature. What has probably happened, and what would you do?

5. Draw a three-level decision tree for deciding whether to carry an umbrella, using at most three questions. Then explain what the tree cannot represent.

PART THREE

The Discipline of Not Fooling Yourself

CHAPTER EIGHT

Overfitting, and the Two Kinds of Being Wrong

A student preparing for an examination has two ways to fail. They can learn too little, arriving with a few general principles and no grip on the specifics, and be unable to answer anything. Or they can learn last year's paper by heart, answering every question on it perfectly and collapsing on anything new.

Machine learning has exactly the same two failures, and they are the two ends of a single dial. Turning the dial towards more flexibility reduces the first and increases the second. The skill is finding the setting that minimises the total, and the instrument that measures it is data the model has not seen.

The U-Shaped Curve

The demonstration is best done with numbers, so here is one. Take twenty points from a smooth curve with a little noise, and fit polynomials of increasing degree.

Model	Parameters	Error on training data	Error on new data
Straight line	2	0.42	0.45
Cubic	4	0.08	0.09
Degree 9	10	0.001	0.35
Degree 15	16	0.0001	2.10

Read the last two rows carefully. The degree-9 model fits the training data almost perfectly and is four times worse on new data than a cubic. The degree-15 model fits the training data better still and is catastrophic.

Nothing went wrong in the fitting. The extra parameters were used exactly as intended: to accommodate the particular positions of twenty noisy points, including the noise. Between the points, and beyond them, the curve swings wildly, because that is what a high-degree polynomial does when asked to pass

through arbitrary points.

This is **overfitting**: the model has learned the sample rather than the pattern.

Two Errors, One Total

The behaviour is best understood by splitting the error into three parts.

$$\text{expected error} = \text{bias}^2 + \text{variance} + \text{irreducible noise}$$

Bias is the error caused by the model being too simple for the truth. A straight line through a curve has high bias, and no amount of data will remove it.

Variance is the error caused by the model being sensitive to the particular sample it was trained on. A degree-15 polynomial has high variance: train it on a different twenty points from the same process and it produces a completely different curve.

Noise is the part of the target that cannot be predicted from the inputs at all. It is the floor, and no model can go below it.

The trade between the first two is the reason the curve in the table is U-shaped. As flexibility rises, bias falls and variance rises. The best model is somewhere in the middle, and where exactly depends on how much data you have: more data reduces variance, so more data allows a more flexible model.

Data size	Model that usually wins
30 rows, 5 features	A simple linear model or a shallow tree
3,000 rows, 20 features	Regularised regression, random forest
300,000 rows, many features	Flexible boosted models, neural networks

The practical rule that follows is uncomfortable and correct: complexity must be paid for with data. A model with a thousand parameters trained on two hundred rows is not ambitious; it is a memory of two hundred rows with a thousand knobs to turn.

Regularisation: Paying for Complexity

There is a way to have a flexible model and still control it: add a penalty for large parameters to the loss.

For a linear model, **ridge regression** minimises

squared error + λ x sum of squared coefficients

The first term wants the model to fit the data. The second wants the coefficients small. The dial λ sets the exchange rate. At λ equal to zero you have ordinary least squares; as λ grows, coefficients shrink towards zero and the model becomes simpler and more stable.

Lasso regression uses the sum of absolute values instead, which has a useful side effect: it drives some coefficients exactly to zero, performing a kind of automatic feature selection.

The effect on the polynomial example is direct. Fitted with a large penalty, the degree-15 model's coefficients are forced small, and the curve becomes smooth again. The flexibility is still there, but it is expensive to use, so it is used only where the data insists.

Regularisation is not magic. It works because it encodes a belief: that large coefficients are usually a sign of fitting noise rather than signal. On a problem where large coefficients are genuinely justified, the penalty fights the truth and must be set small. Choosing λ is, once again, a job for validation data.

Learning Curves

A second diagnostic shows whether more data would help.

Plot error on the vertical axis and training-set size on the horizontal, with two lines: training error and validation error.

If both curves are high and close together, the model is underfitting. More data will not help; a better model will.

If training error is low and validation error is much higher, with a visible gap, the model is overfitting. More data will usually help, and so will regularisation.

If both curves have converged to a low value and close together, the model is working, and the remaining error is close to the noise floor.

Pattern	Diagnosis	Action
Both high, close	Underfitting	More flexible model, better features
Training low, validation high	Overfitting	More data, regularisation, simpler model
Both converging low	Healthy	Stop; measure the floor

Pattern	Diagnosis	Action
Validation worse than training, gap growing	Data quality or leakage	Investigate the split

Stopping Early, on Purpose

There is a regulariser that requires no extra term at all, only patience: stop training before the model finishes learning.

The mechanics are visible in the U-shaped curve. Training error falls monotonically, always. Validation error falls, bottoms out, and rises. The rising phase is not random: the model has begun to fit the particular noise of the training set, and that noise is absent from the validation set. If you stop at the bottom, you keep the general part and discard the memorised part.

Epochs	Training loss	Validation loss
5	0.61	0.58
10	0.44	0.46
20	0.29	0.43
40	0.13	0.49
80	0.04	0.68

The table is a small model on a small dataset, and it is typical in shape. Training loss falls by a factor of fifteen; validation loss falls by a quarter, stops, and then climbs by more than half. The best model is the one at epoch twenty, and everything after that is wasted computation producing a worse system.

The practical implementation is a checkpoint rule: after every epoch, evaluate on the validation set, keep the weights if the score improved, and count how many epochs have passed without improvement. When the count exceeds a patience setting — five or ten epochs is usual — stop and return the best checkpoint rather than the final one.

Two caveats matter.

Early stopping makes the validation set part of training, in a weak sense, because you are choosing a stopping point using it. This is why the test set must be untouched until the end. If you choose the epoch on the test set, the test score is no longer an estimate of anything.

The stopping point is a fitted hyperparameter, and it moves when the data changes. A model that stopped at epoch twenty on last year's data may need thirty on this year's, which is a further argument for automating the choice rather than hard-coding it.

Why This Is the Central Skill

There is a reason this chapter sits in the middle of the volume rather than at the end. Every method in Part Two can be overfitted, and every tool in Part Four exists to detect it. A practitioner who understands the trade can use any library competently; one who does not will report excellent numbers and be surprised in production.

The habit to build is scepticism about good news. When a model reports a dramatic improvement, the first question is not "which feature helped?" but "what did it memorise?". The second question is "what data did I use to measure that, and was it really held out?".

Problems

1. Using the table above, compute the difference between training and test error for each model. Which model would you deploy, and does your answer change if you have ten times as much data?

2. A model has high bias and high variance simultaneously. Is that possible? Explain what it would look like on a learning curve.

3. Ridge regression with $\lambda = 0$, 1 and 100 is applied to a model whose coefficients were 3.0, -2.0 and 0.5. Describe qualitatively what happens to the coefficients as λ rises, and what happens to the predictions on new data.

4. Your model has a training error of 0.02 and a validation error of 0.30. Name three things you would try, in order of likely benefit.

5. Explain in two sentences why adding parameters to a model requires adding data, using the idea of variance.

CHAPTER NINE

Validation, and the Many Ways to Cheat

A team reports 94% accuracy. The number is real: they ran the code, the model produced it, and the test set was genuine. Six months later the system is in production and performing at 78%, and nobody can work out why.

The usual explanation is not fraud. It is that the 94% was measured, repeatedly, while decisions were being made in response to it. Every time the team changed a feature, a threshold, a model or a preprocessing step and remeasured on the same data, a little information from that data entered the model. After fifty such decisions, the test set had quietly become training data.

This chapter is about measuring performance without fooling yourself, which is harder than it sounds and entirely learnable.

The Roles of Three Sets

The split has three members and each has one job.

The **training set** is where parameters are fitted. Nothing else.

The **validation set** is where decisions are made: which model, which features, which threshold, when to stop training. It is looked at many times, and that is fine, because it is being used for its purpose.

The **test set** is opened at the end, once, to produce the number you report. If you ever look at it and then change something in response, it has joined the validation set and you no longer have a test set.

In small projects, one validation set is noisy, and a lucky split can mislead. The standard fix is cross-validation.

Cross-Validation

k-fold cross-validation divides the data into k parts. Train on k-1 of them, test on the remaining one, and repeat until every part has been the test set once. Average the results.

With k equal to five, each model sees 80% of the data for training and 20% for testing, five times. With k equal to ten, 90% and 10%, ten times. Leave-one-out is the extreme: k equals the number of rows.

The choice trades computation against stability. Five-fold is cheap and usually enough. Ten-fold gives a slightly less biased estimate and costs twice as much. Leave-one-out is nearly unbiased and has high variance, which is the opposite of what people expect; it is rarely worth it.

Two refinements matter in practice. **Stratified** folds keep the class proportions the same in each part, which is essential when the rare class is 3% of the data. **Repeated** cross-validation runs the whole procedure several times with different random splits, which gives an honest sense of how much the estimate moves.

And one rule that is broken constantly: all preprocessing, including scaling, imputation and feature selection, must be fitted inside each fold. Washing the whole dataset before splitting leaks information from the test fold into the training fold.

Six Ways to Leak

Leakage is the presence of information in training that will not be available at prediction time. It is the most common cause of a model that looks excellent and fails.

Leak	How it happens	The fix
Target leak	A column that is only filled after the event	Ask what is known at prediction time
Time leak	Random split on time-ordered data	Split by date
Group leak	Same patient or machine in both sets	Split by group
Duplicate leak	Near-identical rows in both sets	De-duplicate, then split
Preprocessing leak	Scaling or imputation fitted on all data	Fit inside each fold

Leak	How it happens	The fix
Selection leak	Choosing features by their correlation with the test set	Select inside each fold

Two of these deserve a story, because they are the ones that survive review.

The target leak is easy to spot once you know to look. A churn model included "number of cancellation emails received". It scored 0.97 AUC and was useless, because a cancellation email is the churn, not a predictor of it. The fix was not statistical; it was asking whether the value would exist for a customer who had not yet cancelled.

The time leak is subtler. A demand model was split randomly, so that some of its training data came from after its test data. It learned the seasonal pattern from the future and appeared to forecast well. Split by date, it lost most of its apparent skill. The fix costs accuracy and buys truth.

A Protocol That Survives Scrutiny

None of this requires a statistician. It requires a sequence that is decided in advance and not changed when the results disappoint.

1. Write down the question, the target, the horizon and the cost of each error.
2. Fix the split before looking at any model. Save the test set aside and do not touch it.
3. Establish the baseline: the mean, the majority class, the previous value.
4. Develop on the training set, decide on the validation set, and keep a log of every decision.
5. When the development stops, run the test set once and report the result, whatever it is.
6. Report the uncertainty, not just the number.

Step five is the one that separates honest work from confident work. A team that opens the test set, dislikes the number, and goes back to development has destroyed its only independent measurement, and no amount of later rigour can restore it.

Reporting Numbers Honestly

A result that can be checked has four parts: the number, the dataset, the uncertainty and the baseline.

"Our model achieved 91% accuracy" is a claim. "Our model achieved 91% accuracy (95% confidence interval 89–93) on 4,000 held-out parts from a single plant, against a majority-class baseline of 88%" is a result. The second costs one extra sentence to write and prevents a year of misunderstanding.

Uncertainty is easy to compute and usually omitted. For a proportion p estimated from n independent cases, the standard error is the square root of p(1-p)/n. With 4,000 cases and p equal to 0.91, that is about 0.0045, so a 95% interval is roughly plus or minus 0.9 percentage points. With 200 cases, the interval is plus or minus four points, and the difference between two models is likely to be noise.

Test cases	Accuracy	95% interval	Difference detectable
100	0.91	± 5.6 points	More than 8 points
1,000	0.91	± 1.8 points	More than 2.5 points
10,000	0.91	± 0.6 points	More than 0.8 points

The table answers a question that consumes enormous amounts of engineering time: how large does a test set need to be to tell two models apart? The answer is usually larger than the one available, which is why small improvements on small datasets should be treated as noise until repeated.

Problems

1. You have 1,200 rows, 8% of them positive, and you want to compare five models. Describe the split and the validation procedure you would use, and say how many times each model is fitted.

2. A colleague scales all features before cross-validation, using the mean and spread of the whole dataset. Explain the leak and estimate whether it would matter more on 200 rows or 200,000.

3. A model predicts equipment failure using a column called "days since last maintenance". Is that a leak? Argue both sides, and say what information you would need.

4. With 500 test cases and 92% accuracy, compute the standard error and a 95% interval. Your colleague's model scores 93.5% on the same test set. Are the models distinguishable?

5. Write the reporting sentence for a result you have produced yourself, with all four parts: number, dataset, uncertainty and baseline.

CHAPTER TEN

Beyond Accuracy

Two defect detectors are offered to a factory. The first catches three quarters of the cracked welds and flags eighty parts a week. The second catches nearly all of them and flags three hundred. Both report similar accuracy. One of them will keep its job; the other will be switched off within a month.

Which one is better depends on what a flag costs and what a missed crack costs, and that is a question about money and risk rather than about mathematics. This chapter is about turning that question into numbers a team can argue with.

The Four Counts

Every binary classifier produces four counts, and everything else is arithmetic on them.

	Predicted positive	**Predicted negative**
Actually positive	True positives	False negatives
Actually negative	False positives	True negatives

From those four, the useful ratios follow.

Precision is true positives divided by everything predicted positive: of the cases we flagged, how many were real?

Recall, also called sensitivity, is true positives divided by everything actually positive: of the real cases, how many did we find?

Specificity is true negatives divided by everything actually negative: of the healthy cases, how many did we leave alone?

F1 is the harmonic mean of precision and recall, a single number that is high only when both are high.

A Worked Comparison

Take a test set of 1,000 welds, 40 of which are cracked.

Model A catches 30 cracks and flags 80 parts in total. Model B catches 38 cracks and flags 300 parts.

	Model A	Model B
True positives	30	38
False negatives	10	2
False positives	50	262
True negatives	910	698
Precision	37.5%	12.7%
Recall	75%	95%
Accuracy	94.0%	73.6%

A has better precision and accuracy. B has better recall. Neither dominates, and the argument between them cannot be settled without costs.

Suppose a missed crack costs 500, because a weld that fails in service is a catastrophe, and a false alarm costs 20, because an inspector must look at the part again.

- Model A: 10 x 500 + 50 x 20 = 5,000 + 1,000 = 6,000.
- Model B: 2 x 500 + 262 x 20 = 1,000 + 5,240 = 6,240.

Model A wins, narrowly, despite finding fewer cracks, because its extra false alarms are cheap and B's are numerous. Change the miss cost to 2,000 and the answer flips: A costs 20,000 + 1,000 = 21,000, B costs 4,000 + 5,240 = 9,240.

This is the whole content of cost-sensitive evaluation. The model does not decide; the costs do. Any team that reports accuracy without costs has decided, silently, that every mistake is equally expensive.

ROC and Its Limits

A classifier's threshold can be moved, trading false positives for false negatives. Sweeping the threshold traces out a curve, and the **receiver operating characteristic**, or ROC, plots the true positive rate against the false positive rate at every threshold.

The **area under that curve**, AUC, summarises it in one number. An AUC of 0.5 means the model is no better than a coin. An AUC of 1.0 means perfect separation. Values around 0.75 are common and useful; 0.9 is excellent; on hard problems, 0.65 may be the state of the art.

AUC has two attractive properties and one serious trap.

The properties: it does not depend on the threshold, and it does not depend on the class balance, because it is computed from rates rather than counts.

The trap: it is dominated by the easy negatives. When the positive class is 1% of the data, a model can achieve a high AUC while being useless at the operating point you care about, because most of the curve is built from cases that were never in doubt.

For rare events, the **precision–recall curve** is more informative. It shows what happens to precision as recall rises, and it makes the cost of catching the last few positives visible. Report both, and report the operating point you will actually use.

Situation	Best summary
Balanced classes, choosing a threshold	ROC and AUC
Rare positives (under 5%), choosing a threshold	Precision–recall curve
A deployed system with fixed costs	Expected cost per decision
Communication with non-specialists	Confusion matrix in counts

Choosing the Threshold

The curve is not only a summary. It is a menu, and the operating point is chosen from it.

Suppose a plant inspects 1,000 parts a day and 30 are defective. The model produces a score, and the threshold converts the score into a decision. Four candidate thresholds, evaluated on a validation set of exactly that composition, give the following.

Threshold	Caught	False alarms	Missed
0.2	26	180	4
0.4	22	70	8
0.6	15	24	15

Threshold	Caught	False alarms	Missed
0.8	7	6	23

No threshold is best in the abstract. The choice depends on what each kind of mistake costs. Say a false alarm costs 60 units — an inspector's time, a re-check, a delayed line — and a missed defect costs 900, because it reaches a customer.

Threshold	Cost of false alarms	Cost of misses	Total
0.2	10,800	3,600	14,400
0.4	4,200	7,200	11,400
0.6	1,440	13,500	14,940
0.8	360	20,700	21,060

The best threshold is 0.4, and it is not the one that maximises any standard metric. It is not the one with the fewest false alarms, nor the one with the most catches. It is the one where the two costs trade evenly at the margin, and it is discoverable only because both costs were written down.

Two disciplines follow from this.

Choose the threshold on validation data, once, and then leave it alone. Adjusting it after seeing test results is a way of importing the test set into the model.

Report the threshold. A precision of 55% is meaningless without the rule that produced it, and the same model at a different threshold would report a different number from the same weights.

There is a useful sanity check available: the cost-optimal threshold sits where the model's scores have the density that balances the two costs. If the chosen threshold falls outside the range where the model produces scores at all, something is wrong with the costs or with the model, and the arithmetic will not save it.

Calibration: Does 0.8 Mean 80%?

A model that outputs probabilities makes a stronger claim than one that outputs scores: it claims that among cases where it says 0.8, about four in five are positive.

Calibration is the agreement between that claim and reality. It is measured by bucketing predictions and comparing the predicted rate with the observed rate in each bucket.

A model can be excellent at ranking, with a high AUC, and badly calibrated. It may order cases correctly and consistently overstate the probability, which matters enormously when the probability feeds a decision rule or a published risk.

Two repairs are standard. **Platt scaling** fits a logistic curve to map scores to probabilities. **Isotonic regression** fits a monotonic step function, which is more flexible and needs more data. Both must be fitted on data the model has not seen, or they will inherit the same overconfidence they are meant to fix.

The **Brier score** summarises calibration and accuracy together: it is the mean squared difference between the predicted probability and the outcome. Lower is better, and it cannot be gamed by a confident, wrong model.

Reporting a Classifier

A useful report contains six things, in this order.

The confusion matrix, in counts, so the reader can compute anything else.

Precision and recall at the operating threshold, with the threshold named.

The baseline: the score of predicting the majority class or the mean.

The curve, ROC or precision–recall as appropriate, so the reader can see what other thresholds would cost.

The calibration, ideally as a small table of buckets.

The uncertainty, as an interval on the headline measure.

Anything less invites the reader to invent the missing parts, and they usually invent them favourably.

Problems

1. A test set has 500 cases, 25 positive. A model flags 40, of which 20 are positive. Build the confusion matrix and compute accuracy, precision, recall and specificity.

2. With a miss cost of 300 and a false alarm cost of 15, compute the expected cost of the model in question 1. Then compute it for a threshold that catches 23 positives with 90 flags. Which is cheaper?

3. Explain in two sentences why a high AUC does not guarantee a good system when the positive class is 1% of the data.

4. A model outputs 0.9 for 100 cases, and 60 of them are positive. Is it calibrated? What would you do about it?

5. Find a reported classifier result — a paper, a product page, a news story — and list which of the six reporting items are present and which are missing.

PART FOUR

From Notebook to System

CHAPTER ELEVEN

Features and the Pipeline That Produces Them

In most tabular projects, the model is a small part of the work. The larger part is turning raw columns — timestamps, category names, missing values, free text, sensor readings at irregular intervals — into a table of numbers that a model can use. That transformation is the **pipeline**, and in practice the pipeline is the model: change it and the model changes, often more than swapping algorithms would.

This chapter is about building pipelines that are correct, reproducible and impossible to leak.

Fit on Train, Apply Forward

Every transformation that learns something from data must be fitted on the training set and then applied, unchanged, to validation, test and production data.

Scaling is the simplest example. Standardisation uses the mean and spread of each feature. If those are computed over the whole dataset, the test set's statistics have entered the model, and the model has seen the future. The effect is small on large datasets and substantial on small ones, and it is a leak regardless.

The same rule governs imputation. A median computed over all rows, including test rows, is a leak. A median computed on training rows and applied everywhere is correct.

The clean way to express this is a pipeline object that bundles the transformations and the model into one unit. Fitting the pipeline fits the transformations on training data only. Predicting applies them in order. There is no opportunity to leak, because there is no separate step in which somebody forgot.

Categories, and the High-Cardinality Trap

Models consume numbers, so categories must be encoded.

One-hot encoding creates a column per category, with a 1 where the category applies. It is simple, interpretable and explodes in size when a category has many values: a postcode with 3,000 distinct values becomes 3,000 columns, almost all zero.

Ordinal encoding assigns each category a number. It is compact and it invents an order, which a linear model will take seriously. It is appropriate for ordered categories such as small, medium and large, and misleading for unordered ones.

Target encoding replaces a category with the average outcome for that category. It is powerful and dangerous: the category's average includes the row being encoded, so the model can see part of its own label. The defences are to compute the encoding inside cross-validation folds, to smooth it towards the global mean for small categories, and to add noise.

Hashing maps categories to a fixed number of buckets. It handles unbounded vocabularies and loses interpretability.

Encoding	Size	Interpretable	Leak risk
One-hot	Grows with categories	Yes	None
Ordinal	One column	Partly	None
Target	One column	No	High without folds
Hashing	Fixed	No	None

Time, Windows and the Shift

Sensor and business data almost always arrives as a time series, and time-series features are where leakage hides most comfortably.

A rolling average is a natural feature: the mean of the last hour's readings. If that average is computed over a window that includes the current row, the row's own value is inside its own feature, and the model will exploit it.

The rule is to use only information available strictly before the moment of prediction. That means shifting the window back by one step, and being explicit about the forecast horizon. A model predicting the next hour cannot use this hour's reading; a model predicting tomorrow can.

Lags are the other workhorse: yesterday's value, last week's value, the value at the same hour seven days ago. They encode the persistence that makes naive forecasts hard to beat, and they are safe as long as the lag is at least as long as the horizon.

Reproducibility, or the Model You Cannot Rebuild

A model that cannot be rebuilt is a liability. Six months after deployment, the team will be asked why it behaves as it does, and the answer must not be "we are not sure which version of the data we used".

Five practices make a project reproducible.

Version the code in a repository, with every experiment as a commit.

Snapshot the data. A hash of the training file is enough to tell you whether the data has changed, and a stored copy is better.

Fix the seed. Random splits, random forests and neural initialisation all depend on the seed; without it, results move between runs and comparisons become noise.

Record the configuration. The model type, hyperparameters, feature list and threshold should be a file, not a memory.

Log the environment. Library versions change behaviour; a model trained under one version may not be reproducible under another.

None of this is exotic. All of it is skipped on small projects, and small projects are exactly where the model will be handed to somebody else.

Features That Carry Meaning

A feature is a hypothesis about what matters, expressed as a number. Good features often beat good algorithms, and they come from understanding the domain rather than from the library.

A maintenance dataset has a vibration column. Useful derived features include the ratio of current vibration to the machine's own baseline, the rate of change over the last week, and the count of days above a threshold in the last month. Each encodes a different belief about how machines fail: gradual drift, sudden change, cumulative exposure.

A retail dataset has transaction timestamps. Useful features include days since last purchase, average interval between purchases, and whether the cus-

tomer bought in the same month last year.

The discipline is to write down, for each feature, the sentence it encodes. If the sentence is nonsense, the feature will be noise. If the sentence is a leak, the model will be brilliant and useless.

Problems

1. List the transformations in a pipeline for a table with numeric columns, a category with 12 values, a date column and a column with 15% missing values. Mark which must be fitted on training data only.

2. A target encoding gives a category with 3 rows an average of 1.0 and a category with 300 rows an average of 0.4. Which is more trustworthy, and how would you smooth it?

3. You build a feature "average sales in the last 7 days" for a model that predicts tomorrow's sales. Explain the shift you must apply and what happens if you forget.

4. A colleague cannot reproduce your model's score from last week. Name five things that might have changed and how you would detect each.

5. Invent three features for a prediction task you know well, and write the sentence each one encodes.

CHAPTER TWELVE

Deployment, Drift and the Long Middle

A model is a snapshot of the world as it was when the data was collected. Deployment puts that snapshot in front of a world that keeps moving. The moment between the end of development and the first month in production is where most of the value of a machine learning project is either captured or lost, and it is the part that courses rarely teach.

This chapter is about the long middle: the years in which a model is running, quietly decaying, and either being maintained or being trusted past its usefulness.

What Deployment Changes

Three things change the moment a model starts making decisions.

The population changes. A model trained on last year's customers meets this year's, and the new ones are different precisely because the model exists. A retention offer changes who stays; a fraud model changes what fraudsters attempt; a screening tool changes who is tested.

The measurement changes. In development, labels arrive in a tidy file. In production, labels arrive late, if at all: the failure is confirmed next month, the customer cancels next quarter, the diagnosis comes back from the laboratory. A system that cannot wait for labels must be monitored indirectly.

The consequences change. A wrong prediction in a notebook is a wrong number. A wrong prediction in production is a blocked payment, a delayed delivery or an unnecessary inspection, and somebody has to deal with it.

Getting It Into Service

Two deployment patterns reduce the risk of a bad first month.

A **shadow deployment** runs the new model alongside the old one, without acting on its output, and compares their predictions on live data. It costs computation and nothing else, and it catches mismatches between the training pipeline and the production pipeline before any customer sees them.

A **canary release** sends a small share of traffic to the new model, perhaps 5%, and compares outcomes against the rest. It is the only way to measure the effect of the model on the thing you actually care about, which is rarely the model's accuracy.

Both patterns share a principle: never let a model's first contact with reality be its full workload.

Three Signals to Watch

A monitoring system needs to watch inputs, outputs and outcomes, because the three fail differently.

Inputs. For each feature, track the mean, the spread, the missing rate and the distribution. Compare against the training distribution. A shift in a feature's mean by more than a fraction of its training spread is worth an alert, because it usually means something upstream changed: a sensor replaced, a unit converted, a form redesigned.

Outputs. Track the rate of positive predictions, the average score, the distribution of scores. A fraud model that suddenly flags 4% of transactions instead of 0.5% has either found something or broken, and both require a human.

Outcomes. When labels arrive, compute the real metrics. This is the only direct measure of performance, and it is always delayed. Until it arrives, the input and output signals are proxies.

Population stability is often summarised with a single number, the **population stability index**, which compares the distribution of a feature in production with its distribution in training. A value below 0.1 is usually unremarkable; above 0.25 is a signal to investigate. The thresholds are conventions, not laws, and the number should be used to prioritise attention rather than to trigger automatic retraining.

Three Kinds of Drift

The word drift covers three distinct problems, and they need different responses.

Covariate shift is a change in the inputs. The relationship between input and output is unchanged, but the inputs are different: a new camera, a different season, a new customer segment. Retraining on recent data usually fixes it.

Label shift is a change in the proportion of classes. Fraud rises from 0.2% to 1%. The model may still rank correctly, but any fixed threshold now produces far more alerts, and the threshold must move.

Concept drift is a change in the relationship itself. What predicted repayment in 2006 did not predict it in 2009. Retraining on recent data helps only if the new relationship is stable; if the world is changing every month, no model will stay useful for long, and the honest response is to shorten the review cycle.

Kind	What changes	First symptom	Response
Covariate	Input distribution	Feature drift alerts	Retrain on recent data
Label	Class proportions	Alert rate jumps	Recalibrate the threshold
Concept	Input–output relation	Accuracy falls with labels	Investigate, retrain, shorten cycles

Retraining Without Chaos

A retraining policy is a business process as much as a technical one. It needs three things: a trigger, a gate and a rollback.

The **trigger** is either scheduled, such as quarterly, or conditional, such as an alert on outcome metrics. Scheduled retraining is simple and can waste effort; conditional retraining reacts but can be gamed by whoever sets the threshold.

The **gate** is a comparison on held-out data: the new model must beat the incumbent on the metric that matters, with an allowance for uncertainty. A model that is 0.3 percentage points better on 400 test cases has not been shown to be better at all.

The **rollback** is a stored previous version and a one-command route back to it. Every system that cannot roll back is one bad retraining away from an outage.

Two further practices are worth the effort. Version the data as well as the code, so that a model can be rebuilt exactly. And write a short model card for each release: what it predicts, on what population, with what measured performance, what it must not be used for, and who owns it.

The Feedback Loop

The most dangerous property of a deployed model is that it changes the data it will be trained on next.

A model that predicts which machines will fail sends inspectors to those machines. The other machines are inspected less, so their failures are recorded later or not at all. The next training set is therefore biased towards the machines the model already flagged, and the model's apparent accuracy improves while its coverage of the fleet quietly narrows.

The same pattern appears in hiring, lending and policing, and it is not a statistical curiosity. It is the system's own success corrupting its evidence.

The defences are deliberate: keep a random sample of cases that the model did not choose, record outcomes for them, and include them in evaluation. A small control group, chosen at random and treated identically, is often the only honest way to know whether a deployed model is helping.

Alert Fatigue, in Numbers

A model's output is not the end of the system. Someone receives it, and the arithmetic of what they receive determines whether the system survives its first month.

Take a plant performing 2,000 welds a day with a 1.5% defect rate: 30 real defects. The detector catches 85% of them, which is good, and its alerts are right 22% of the time, which is ordinary for a rare event.

Quantity	Value
Welds per day	2,000
Real defects	30
Caught	26
Alerts raised	118
False alarms	92

Each inspector can properly review 40 alerts per shift. On paper, 118 alerts is three shifts of work, and the plant has one. In practice the queue does something worse than overflow: it teaches people to dismiss.

Within a few weeks the pattern is measurable. Alerts reviewed falls to 60 a day, then 40. The proportion dismissed without inspection climbs past half. Effective recall — the fraction of the 30 defects actually acted on — falls to around 50%, and the model's headline 85% becomes a number that describes nothing anyone does.

There are four repairs, and they are usually applied together.

Raise the threshold, accepting fewer catches in exchange for alerts people believe. Precision of 22% is not a fact of nature; it is a choice of operating point.

Route the cheap cases elsewhere. A low-score part can be inspected later by sampling rather than immediately by a person.

Cap the queue explicitly and report the overflow, so that the system's silence is visible rather than mistaken for safety.

Measure the response, not only the prediction. Log which alerts were reviewed and what the reviewer found. That log is the only honest measure of whether the model is doing anything.

The general lesson is that precision and recall are properties of a model, while workload and trust are properties of a system. A deployment plan that computes only the first pair is planning half a system.

Problems

1. Design a monitoring plan for a model that predicts equipment failure. Name three input signals, two output signals and the outcome metric, and say how often each is checked.

2. A feature's mean shifts by 0.8 standard deviations. List three possible causes and the check that would distinguish them.

3. Your new model is 1.2 percentage points better on a test set of 300 cases. Compute the standard error of the difference approximately, and decide whether to deploy.

4. Describe the feedback loop for a model that decides which students receive tutoring. How would you collect evidence about the students the model did not choose?

5. Write a model card for a model you have built or used: purpose, population, performance, limitations, owner, review date.

CHAPTER THIRTEEN

When Machine Learning Is the Wrong Tool

A team is asked to build a model that decides whether a payment should be blocked. The rule that governs the decision is written in regulation, and it must be applied identically to every customer, in a fixed order, with a reason recorded. Nobody wants a model for that. They want the rule, implemented carefully, with an audit trail.

Knowing when not to use machine learning is a mark of competence, and it is rarely taught because the enthusiasm of the field runs the other way. This chapter is a set of questions to ask before the first line of code.

Four Questions

Is the pattern real and stable? If the relationship between inputs and outcomes changes faster than the model can be retrained, learning is a treadmill. Some domains are stable — the physics of a bearing, the optics of a lens — and some are not, such as fashion, fraud and markets. Ask how long the relationship has held and what would break it.

Is there enough data, of the right kind? Enough means enough examples of the thing you are predicting, not enough rows in total. Forty machines and six failures is not a dataset for a flexible model; it is a dataset for a rule written by the person who maintains the machines.

Does the decision tolerate being wrong in a measurable way? A model that is 85% accurate is useful when the cost of mistakes is small and recoverable, and unacceptable when a single mistake is fatal and irreversible.

Can the decision be explained to the person it affects? In credit, employment, insurance and medicine, laws increasingly require a reason. A model can sometimes supply one, but a rule always can.

If any of these answers is uncomfortable, the simpler alternative is usually better, and it is always cheaper.

The Alternative Toolkit

Machine learning is one tool among several, and the others are often better.

Rules are the right choice when the knowledge is documented, the decision must be auditable, or the cost of an error is extreme. A safety interlock, a tax computation, a clinical protocol. Rules are cheap to run, easy to test exhaustively and easy to change.

Lookup tables and thresholds handle a surprising share of industrial decisions. A part is rejected if its measurement is outside a tolerance set by engineering, and the tolerance is derived from the function of the part, not from a dataset.

Optimisation is the right tool when the question is "what is the best allocation" rather than "what will happen". Scheduling, routing and packing are optimisation problems, and a model that predicts demand feeds the optimiser rather than replacing it.

Simulation answers "what happens if" when no data exists for the situation, as it often does not for a machine that has never failed.

Statistical process control detects that something has changed in a process, using a handful of numbers and a chart, and it does so with a transparency that a model cannot match.

A person is the right answer when the decision requires judgement, context or accountability that cannot be delegated. The useful question is not whether to automate the whole decision, but which part of it to automate.

Situation	Better tool than ML
Decision must be audited line by line	Rules
Only a few dozen examples exist	Rules or expert judgement
Question is how to allocate a fixed resource	Optimisation
Situation has never occurred	Simulation
Need to know a process has shifted	Control charts
Decision carries legal accountability	Human, with ML as advice

The Case of the Tiny Dataset

There is a recurring fantasy that a clever model can extract a pattern from almost no data. Sometimes, with strong prior knowledge and a very small model, something can be learned. Usually what happens is that the model memorises the examples and the team mistakes this for insight.

A useful test: hold out two examples out of ten and see whether the model predicts them. It usually does, and the result is meaningless, because the model saw eight and was fitted until it agreed with them.

With tiny data, the honest approaches are: use a model with two or three parameters, such as a threshold or a line; use domain knowledge to constrain the model; collect more data before modelling; or accept that the decision is a judgement call and support it with a transparent calculation rather than a fitted model.

The Case of Causation

A model trained on observational data learns associations. When the decision changes the world, the association may not survive the change.

A hospital observes that patients who receive a particular test have worse outcomes, and concludes that the test is harmful. In fact, doctors order the test for patients they are worried about, so the test is a marker of severity rather than a cause. A model that recommends reducing the test would be confidently wrong, and the harm would fall on the patients the doctors were right to worry about.

The question to ask before acting on any model is: if I intervened and changed this input, would the outcome change, and in which direction? Without an experiment or a causal model, the honest answer is often "we do not know", and a decision that changes people's lives should not be built on an unknown.

Model Plus Rule

The most common industrial arrangement is not a model instead of a rule. It is a model and a rule, each doing the part it is good at.

Consider a weld inspection cell. A vision model scores each image for porosity. A geometric check measures bead width and flags anything outside a

tolerance that the process engineer can justify from first principles. Neither is a substitute for the other: the model sees texture the rule cannot express, and the rule sees a physical limit the model has never been taught to respect.

The combination has a shape worth naming: the model proposes, the rule constrains.

Configuration	Recall	Precision
Rule alone	60%	48%
Model alone	88%	30%
Model, then rule as veto	80%	55%

The third row costs eight points of recall and gains twenty-five points of precision, which is usually the better trade, because the veto removes exactly the class of error people find least forgivable: confident nonsense about a part that is visibly wrong by measurement.

Three design details make the pattern work.

The rule must be independently justified. If the geometric tolerance was itself tuned against the same labels, the combination is one model wearing two hats, and the agreement between them is meaningless.

The veto must be logged. Every case where the rule overrides a high model score is a case where the model was wrong about something checkable, and a run of them is the earliest signal of drift available.

The arrangement must fail safe. If the camera fails, the rule must still run, and the line must stop rather than pass parts on the strength of a missing score.

The same pattern appears far from factories. A language model proposes a reply and a rule blocks it if it contains an account number. A credit model scores an applicant and a policy declines anyone with a recent default. In every case the rule is doing work the model cannot do reliably — enforcing a commitment that is not in the training data — and the model is doing work the rule cannot express.

A Closing Checklist

Before building, answer these six questions in writing. They take twenty minutes and save months.

What decision will this model change, and who makes that decision today?

What is the target, in one sentence, with a horizon and a unit?

What would I know at prediction time, and does the data contain any of it by accident?

How many examples of the outcome do I have, and is that enough for the model I intend to use?

What does each kind of mistake cost, and what threshold follows?

If this model works perfectly, how would the world change, and who would notice it was wrong?

If any answer is "we will work that out later", the project is not ready. The later is where the budget goes.

Problems

1. For each decision, choose between machine learning, a rule, optimisation, simulation, or a human: rejecting a part outside tolerance; deciding a delivery route; approving a mortgage; predicting demand for a new product; deciding whether a machine needs maintenance; deciding whether a patient needs surgery. Justify each in one sentence.

2. You have 60 labelled examples and 12 features. Write the argument you would make to a manager who wants a gradient-boosted model, and the alternative you would propose.

3. Construct a plausible example, from a domain you know, where a model's input is caused by the outcome rather than causing it. Explain how the mistake would show up in deployment.

4. Write the six-question checklist for a project you would like to do, answering each honestly. If you cannot answer one, say what you would need to find out.

Glossary

Accuracy — the fraction of predictions that are correct. Meaningless without a baseline.

Active learning — labelling only the examples the model is least certain about, to reduce labelling cost.

AUC — the area under the ROC curve; the probability that a randomly chosen positive is ranked above a randomly chosen negative.

Baseline — the score of a trivial method, such as the mean or the majority class.

Bias (statistical) — error caused by a model too simple to represent the truth.

Bias–variance trade — the balance between error from oversimplification and error from sensitivity to the training sample.

Brier score — the mean squared difference between predicted probability and outcome; lower is better.

Calibration — agreement between predicted probabilities and observed frequencies.

Class imbalance — a dataset in which one class is much rarer than another.

Confusion matrix — a table of predicted against actual classes.

Concept drift — a change in the relationship between inputs and outputs.

Covariate shift — a change in the distribution of inputs, with the relationship unchanged.

Cross-entropy — the loss used for classification, equal to minus the logarithm of the probability assigned to the true class.

Cross-validation — repeatedly splitting the data into training and held-out parts so that every case is tested once.

Curse of dimensionality — the loss of meaning of distance, and the growth in data requirements, as the number of features rises.

Decision tree — a model that splits data by a sequence of yes-or-no questions.

Discriminative model — a model that predicts the target directly, rather than modelling the data's distribution.

Ensemble — a model built by combining several models.

Feature — a measurable input variable used by a model.

Feature importance — a measure of how much a feature contributes to a model's predictions.

F1 score — the harmonic mean of precision and recall.

Gini impurity — a measure of class mixing in a group, used to choose splits in trees.

Gradient boosting — an ensemble built by adding trees that correct previous errors.

Hyperparameter — a setting chosen before training, such as tree depth or regularisation strength.

Imputation — filling missing values with a substitute.

Inductive bias — the assumptions a model makes that let it generalise from finite data.

Label — the known answer for a training example.

Label shift — a change in the proportion of classes in the population.

Lasso — regression with a penalty on the absolute size of coefficients, which can set some to zero.

Leakage — information in training that will not be available at prediction time.

Logistic regression — a linear model passed through the logistic function to produce probabilities.

Loss — the single number a model minimises during training.

Overfitting — fitting the training sample so closely that new data suffers.

Pipeline — the ordered set of transformations and the model, fitted and applied as one unit.

Precision — of the cases predicted positive, the fraction that are correct.

Precision–recall curve — precision plotted against recall across thresholds; preferred for rare positives.

Random forest — an ensemble of decision trees trained on random samples and feature subsets.

Recall — of the actual positives, the fraction that were found.

Regression — predicting a numeric value.

Regularisation — a penalty on model complexity used to reduce variance.

Residual — the difference between prediction and truth for one example.

ROC curve — true positive rate against false positive rate across thresholds.

Stratified split — a split that preserves class proportions in each part.

Supervised learning — learning from examples with known answers.

Target encoding — replacing a category with the average outcome for that category.

Underfitting — a model too simple to capture the pattern.

Unsupervised learning — finding structure in data without labels.

Validation set — data used to make modelling decisions, never for final reporting.

Variance — error caused by sensitivity to the particular training sample.

Sources and Further Reading

The methods in this volume are standard, and the sources below are the ones worth reading next. Where a number in the text is an illustrative example rather than a measurement, the text says so.

Textbooks

Hastie, T., Tibshirani, R., and Friedman, J. *The Elements of Statistical Learning*. Free online. The reference for regression, regularisation, trees and validation, written with unusual care about how models fail.

Bishop, C. *Pattern Recognition and Machine Learning*. The probabilistic view, and the best account of logistic regression as a model rather than a formula.

Géron, A. *Hands-On Machine Learning with Scikit-Learn, Keras and TensorFlow*. The practical companion; use it for the pipeline and deployment chapters.

Murphy, K. *Machine Learning: A Probabilistic Perspective*. Comprehensive and demanding; useful once the intuitions here are firm.

Kuhn, M., and Johnson, K. *Feature Engineering and Selection*. A careful treatment of the part of the work that most determines the result.

Papers worth reading in full

Breiman, L. (2001). “Random Forests.” *Machine Learning*. The original, and clearer than most explanations of it.

Breiman, L. (2001). “Statistical Modeling: The Two Cultures.” A short, argumentative paper about the difference between explaining and predicting, and why both matter.

Sculley, D., et al. (2015). “Hidden Technical Debt in Machine Learning Systems.” The paper that named the problem of maintaining models in production.

Gebru, T., et al. (2018). “Datasheets for Datasets.” A one-page template that prevents most data surprises.

Mitchell, M., et al. (2019). “Model Cards for Model Reporting.” The template for describing a deployed model honestly.

Ribeiro, M., Singh, S., and Guestrin, C. (2016). “Why Should I Trust You?” The paper behind LIME, and the husky-and-snow analysis.

On evaluation and its failures

Kaufman, S., Rosset, S., Perlich, C., and Stitelman, O. (2012). “Leakage in Data Mining: Formulation, Detection, and Avoidance.” The systematic treatment of the subject that this volume's chapter on validation summarises.

Hernán, M., Hsu, J., and Healy, B. (2019). “A Second Chance to Get Causal Inference Right.” A readable argument for why prediction and causation are different questions.

Practice and judgement

Domingos, P. *The Master Algorithm*. A survey of the field's main traditions, useful for orientation.

O'Neil, C. *Weapons of Math Destruction*. Cases in which models were deployed without the checks this volume describes.

Rule, A., et al. (2019). “Ten Simple Rules for Reproducible Research in Computational Science.” Short, practical, and applicable well beyond science.

All computations in this volume can be reproduced with a calculator. The datasets are invented for teaching, and the numbers are chosen to be checkable by hand.